TrustVault: API Documentation

API | Snovasys Fresher Hiring Challenge 2026

1. Overview

TrustVault has API endpoints for login, document management, sharing with permissions, downloading, and reading the audit log.

The base address of the deployed application is https://trustvault-lh94.onrender.com. Requests and answers use JSON, except where an endpoint sends back a document file.

Protected endpoints need a JSON Web Token (JWT) in the Authorization header:

```
Authorization: Bearer <JWT_TOKEN>
```

The exact request fields and answers are defined in the code of the repository, and that code is the final reference.

2. Access levels and roles

Level	Meaning
Public	No login is needed.
Authenticated user	A user with a valid token.
Document owner	The user who owns the document.
Authorized user	The owner, or a user who was given access through document permissions.
Administrator	A user whose role allows admin operations, such as reading the audit log.

The backend checks permission on every protected request. Knowing a document ID does not give access.

Demo accounts

These accounts are on the live site so that the endpoints can be tried:

Account	Email	Password
Admin	23jr1a05h4@gmail.com	snovasys@123
User 1	bhanu@gmail.com	bhanu@123
User 2	prakash@gmail.com	prakash@123

The admin account has the administrator role. User 1 and User 2 are normal accounts: a normal user can upload documents, owns what they upload, and can share their own documents with the other user as read or edit. To log in, send the email and the password to the login endpoint, take the token from the answer and send it in the Authorization header.

3. Health check

Item	Detail
Endpoint	GET /api/health
Who can call it	Public
Purpose	Shows that the backend is running and reports the health of the application and the database connection.
Answer	A JSON object with the service status and the database connection status.

Example: GET https://trustvault-lh94.onrender.com/api/health

4. Login endpoints

Endpoint	Who can call it	What it does
POST /api/auth/register	Public, if registration is enabled	Creates an account. The data is validated, the password is hashed before saving, duplicate emails are refused, and the password hash is never returned.
POST /api/auth/login	Public	Checks the email and password and returns a JWT if they are correct. An invalid login gets an error, and the password hash is never returned.
GET /api/auth/me	Any logged-in user	Returns information about the user who owns the token, and nothing about other users.

Before the final submission I will check the register route and its fields in the auth routes file, and remove it from this document if it is not enabled.

5. Document endpoints

This table shows who may use each document action. The exact HTTP method and path of each one are defined in server/routes/documents.js, and I must copy them from that file into this table before submission.

Action	Who can call it	Rules enforced by the server
Upload a document	Any logged-in user	The owner is set by the server from the token and not from the request. File type and size are validated. The file is saved in private storage.
List documents	Any logged-in user	Returns only documents the user owns or has permission for.
Read a document	Owner, or a user with read permission	Checks that the document exists, then checks ownership or permission, and refuses everyone else.

Action	Who can call it	Rules enforced by the server
Download a document	Owner, or a user with permission	Checks the permission before the file is touched. The real storage path is never shown. Success and denied attempts are written to the audit log.
Grant permission	Owner, or another role that the code explicitly allows	Checks that the caller can manage the document, validates the target user and the permission level.
Revoke permission	Owner, or another role that the code explicitly allows	Checks that the caller can manage the document, and that the permission belongs to that document and that user. It takes effect on the next request.

Errors that these endpoints can give:

- 401 Unauthorized: the token is missing or not valid.
- 403 Forbidden: the user is logged in but does not have permission.
- 404 Not Found: the document or its stored file does not exist.

6. Admin endpoint

Item	Detail
Endpoint	GET /api/admin/audit-logs
Who can call it	Administrator only
Purpose	Returns the audit records so that the admin can monitor activity and review security events.
Rules	A valid token is required. The role is checked on the server. A request with no token is refused, and a request from a non-admin user is refused.
Errors	401 if there is no valid login. 403 if the user is not an administrator.

7. Status codes

Code	Meaning
200	The request worked.
201	Something was created, where an endpoint uses it.
400	The request is wrongly formed or has invalid data.
401	Login is missing or not valid.
403	The user is logged in but not allowed.
404	The document or the stored file was not found.
413	The upload is too large, where the application handles it.
415	The file type is not accepted, where the application handles it.
500	An unexpected error happened on the server.

The real code depends on how each route handles its errors.

8. Security design

- Authentication: the JWT tells the server who is calling.
- Authorization: the backend checks ownership, permissions and the admin role, on every request.
- Private storage: uploaded files are not served from a public folder.
- Audit log: important actions and denied attempts are recorded where the audit code covers them.
- Validation: requests and uploads are checked before they are used.
- Least privilege: a user gets only the access needed for what they are allowed to do.

9. Security testing of the API

These are the things that I test against the API:

- A missing or invalid token.
- Opening another user's private document.
- Downloading a document without permission.
- Changing permissions without being allowed to.
- A normal user calling the admin endpoint.
- Sharing and access that is allowed, which must work.
- A missing document and a missing stored file.
- The audit entries that these actions should create.

10. Keeping this document correct

Before I submit I go through this list:

1. Compare every endpoint here with the Express route definitions.
2. Confirm the HTTP method and the path of each one.
3. Confirm the request fields.
4. Confirm the answers and the status codes.
5. Delete anything that is not implemented.
6. Update this file whenever the API changes.

11. Conclusion

The TrustVault API is built around login, permission checks on each document, role limits and an audit trail. The backend enforces all of this by itself, so that a direct API request cannot go around the document permissions even if the frontend is skipped.